Rcode

```
library(ggplot2)
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.1.4     ✔ readr     2.1.5
✔ forcats   1.0.0     ✔ stringr   1.5.1
✔ lubridate 1.9.5     ✔ tibble    3.3.0
✔ purrr     1.1.0     ✔ tidyr     1.3.1
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
become errors
```

```
library(rstan)
```

```
Loading required package: StanHeaders

rstan version 2.32.7 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling
options(mc.cores = parallel::detectCores()).
To avoid recompilation of unchanged Stan programs, we recommend calling
rstan_options(auto_write = TRUE)
For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
change `threads_per_chain` option:
rstan_options(threads_per_chain = 1)


Attaching package: 'rstan'

The following object is masked from 'package:tidyr':

    extract
```

```
curlin <- read_csv("curling.csv")
```

```
Rows: 311 Columns: 6
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): CompetitionName
dbl (4): MatchID, EndID, USWin, Points
lgl (1): PowerPlay

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#Potential data cleaning / Priors from Beijing
Beijing <- curlin %>%
  filter(CompetitionName == "Beijing 2022 Olympic Winter Games")
mean(Beijing$Points) #3.22 average score of intentions
```

```
[1] 3.222222
```

```
Powerplay <- Beijing %>% filter(PowerPlay == T)
length(Beijing$PowerPlay) /
  length(Powerplay$PowerPlay)
```

```
[1] 8
```

```
# 0.125 Powerplays per year
# (Dumb code and dumber user pp per year doesnt even matter)

curlin_excised <- curlin %>%
  filter(!(CompetitionName %in% "Beijing 2022 Olympic Winter Games"))
```

```
# check how many observations we should have (32)
curlin %>%
  filter(!str_detect(CompetitionName, "Beijing")) %>%
  group_by(CompetitionName) %>%
  summarize(num_matches = max(MatchID))
```

```
# A tibble: 3 × 2
  CompetitionName                                num_matches
  <chr>                                                <dbl>
1 World Mixed Doubles Curling Championship 2023           12
2 World Mixed Doubles Curling Championship 2024            9
3 World Mixed Doubles Curling Championship 2025           11
```

```
# create a new variable for successful shot
# (deemed a Points score of higher than 2)
curlin <- curlin %>% mutate(Success = ifelse((Points > 2), 1, 0))

successful_powerplays <- curlin %>%
  # filter to competitions other than Beijing Olympics, consider only powerplays
  filter(!str_detect(CompetitionName, "Beijing"),
         PowerPlay) %>%
  # create a PowerPlaySuccessful variable
  mutate(PowerPlaySuccessful = if_else((Success == 1), TRUE, FALSE)) %>%
  select(CompetitionName, MatchID, PowerPlaySuccessful)

n_successful_non_powerplays <- curlin %>%
  # filter to competitions other than Beijing Olympics,
  # consider only non powerplays
  filter(!str_detect(CompetitionName, "Beijing"),
```

```
        !PowerPlay) %>%
  select(!c(PowerPlay, Points)) %>%
  group_by(CompetitionName, MatchID) %>%
  # count how many successful non powerplays there were per match
  summarize(SuccessfulNonPowerPlays = sum(Success))
```

```
`summarise()` has grouped output by 'CompetitionName'. You can override using
the `.groups` argument.
```

```
final_curling <- curlin %>%
  # filter to competitions other than Beijing Olympics
  filter(!str_detect(CompetitionName, "Beijing")) %>%
  select(!Success) %>%
  # pivot so there's one row per match.
  # information about powerplay and points go to a column per end
  pivot_wider(
    names_from = "EndID",
    values_from = c("PowerPlay", "Points"),
    values_fill = list("PowerPlay" = FALSE, "Points" = 0)
  ) %>%
  # determine if there's at least one powerplay in a match
  mutate(
    PowerPlayPresent =
      if_else((PowerPlay_1 | PowerPlay_2 | PowerPlay_3 | PowerPlay_4 |
                PowerPlay_5 | PowerPlay_6 | PowerPlay_7 | PowerPlay_8 |
                PowerPlay_9), TRUE, FALSE)
  ) %>%
  # add on information regarding successful powerplays
  left_join(successful_powerplays,
            by = join_by("CompetitionName" == "CompetitionName",
                         "MatchID" == "MatchID")) %>%
  # add on information regarding number of successful non powerplays
  left_join(n_successful_non_powerplays,
            by = join_by("CompetitionName" == "CompetitionName",
                         "MatchID" == "MatchID")) %>%
  # turn NA values from lack of powerplay in match to FALSE,
  # create index for competition name
  mutate(PowerPlaySuccessful =
           if_else(is.na(PowerPlaySuccessful), FALSE, PowerPlaySuccessful),
         CompetitionIndex = case_when(
           CompetitionName == "World Mixed Doubles Curling Championship 2023" ~ 1,
           CompetitionName == "World Mixed Doubles Curling Championship 2024" ~ 2,
           CompetitionName == "World Mixed Doubles Curling Championship 2025" ~ 3
         ))
```

```
final_curling %>%
  summarize_at(vars(starts_with("PowerPlay_")), mean)
```

```
# A tibble: 1 × 9
  PowerPlay_1 PowerPlay_2 PowerPlay_3 PowerPlay_4 PowerPlay_5 PowerPlay_6
```

```
        <dbl>       <dbl>       <dbl>       <dbl>       <dbl>       <dbl>
1           0           0           0           0      0.0625       0.188
# ℹ 3 more variables: PowerPlay_7 <dbl>, PowerPlay_8 <dbl>, PowerPlay_9 <dbl>
```

```
successful_powerplays_beijing <- curlin %>%
  # filter to competitions other than Beijing Olympics, consider only powerplays
  filter(str_detect(CompetitionName, "Beijing"),
         PowerPlay) %>%
  # create a PowerPlaySuccessful variable
  mutate(PowerPlaySuccessful = if_else((Success == 1), TRUE, FALSE)) %>%
  select(CompetitionName, MatchID, PowerPlaySuccessful)

n_successful_non_powerplays_beijing <- curlin %>%
  # filter to competitions other than Beijing Olympics,
  # consider only non powerplays
  filter(str_detect(CompetitionName, "Beijing"),
         !PowerPlay) %>%
  select(!c(PowerPlay, Points)) %>%
  group_by(CompetitionName, MatchID) %>%
  # count how many successful non powerplays there were per match
  summarize(SuccessfulNonPowerPlays = sum(Success))
```

```
`summarise()` has grouped output by 'CompetitionName'. You can override using
the `.groups` argument.
```

```
final_curling_beijing <- curlin %>%
  # filter to competitions other than Beijing Olympics
  filter(str_detect(CompetitionName, "Beijing")) %>%
  select(!Success) %>%
  # pivot so there's one row per match.
  # information about powerplay and points go to a column per end
  pivot_wider(
    names_from = "EndID",
    values_from = c("PowerPlay", "Points"),
    values_fill = list("PowerPlay" = FALSE, "Points" = 0)
  ) %>%
  # determine if there's at least one powerplay in a match
  mutate(
    PowerPlayPresent = if_else((PowerPlay_1 | PowerPlay_2 | PowerPlay_3 |
                                PowerPlay_4 | PowerPlay_5 | PowerPlay_6 |
                                PowerPlay_7 | PowerPlay_8 | PowerPlay_9),
                               TRUE, FALSE)
  ) %>%
  # add on information regarding successful powerplays
  left_join(successful_powerplays_beijing,
            by = join_by("CompetitionName" == "CompetitionName",
                         "MatchID" == "MatchID")) %>%
  # add on information regarding number of successful non powerplays
  left_join(n_successful_non_powerplays_beijing,
            by = join_by("CompetitionName" == "CompetitionName",
```

```
                          "MatchID" == "MatchID")) %>%
  # turn NA values from lack of powerplay in match to FALSE, create index for competition
  mutate(PowerPlaySuccessful =
           if_else(is.na(PowerPlaySuccessful), FALSE, PowerPlaySuccessful),
         CompetitionIndex = case_when(
           CompetitionName == "World Mixed Doubles Curling Championship 2023" ~ 1,
           CompetitionName == "World Mixed Doubles Curling Championship 2024" ~ 2,
           CompetitionName == "World Mixed Doubles Curling Championship 2025" ~ 3
         ))
```

```
final_curling_beijing %>%
  group_by(PowerPlayPresent) %>%
  summarize(win_percentage = mean(USWin))
```

```
# A tibble: 1 × 2
  PowerPlayPresent win_percentage
  <lgl>                     <dbl>
1 TRUE                      0.333
```

Partial Pooling for Q1 (DO NOT USE):

```
q1_prior <- glm(USWin ~ PowerPlay_5 + PowerPlay_6 + PowerPlay_7 + PowerPlay_8,
                data = final_curling_beijing, family = "binomial")

q1_prior %>%
  summary()
```

```
Call:
glm(formula = USWin ~ PowerPlay_5 + PowerPlay_6 + PowerPlay_7 +
    PowerPlay_8, family = "binomial", data = final_curling_beijing)

Coefficients: (1 not defined because of singularities)
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)     -1.957e+01  1.075e+04  -0.002    0.999
PowerPlay_5TRUE -3.017e-08  1.521e+04   0.000    1.000
PowerPlay_6TRUE -3.016e-08  1.521e+04   0.000    1.000
PowerPlay_7TRUE  1.957e+01  1.075e+04   0.002    0.999
PowerPlay_8TRUE         NA         NA      NA       NA

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 11.4573  on 8  degrees of freedom
Residual deviance:  8.3178  on 5  degrees of freedom
AIC: 16.318

Number of Fisher Scoring iterations: 18
```

```r
ppq_1_data <- list(N = length(final_curling$CompetitionName), # number of matches
                   k = length(unique(final_curling$CompetitionName)), # Number of competi
                   PowerPlay_5 = final_curling$PowerPlay_5,
                   PowerPlay_6 = final_curling$PowerPlay_6,
                   PowerPlay_7 = final_curling$PowerPlay_7,
                   PowerPlay_8 = final_curling$PowerPlay_8,
                   y = final_curling$USWin,
                   i = final_curling$CompetitionIndex, # for indexings sake
                   u0 = 0,
                   u1 = 0,
                   u2 = 0,
                   u3 = 0,
                   u4 = 0,
                   v0 = 1,
                   v1 = 1,
                   v2 = 1,
                   v3 = 1,
                   v4 = 1,
                   alpha0 = 1,
                   alpha1 = 1,
                   alpha2 = 1,
                   alpha3 = 1,
                   alpha4 = 1,
                   eta0 = 1,
                   eta1 = 1,
                   eta2 = 1,
                   eta3 = 1,
                   eta4 = 1
)


set.seed(454)

fit <- stan(file = "partial_pooling_q1.stan",
            data = ppq_1_data,
            chains = 4,
            warmup = 1000,
            iter = 2000,
            control = list(adapt_delta = 0.999))
```

```
Trying to compile a simple C file

Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
using C compiler: ‘Apple clang version 17.0.0 (clang-1700.6.3.2)’
using SDK: ‘MacOSX26.2.sdk’
clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -
DNDEBUG   -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/Rcpp/include/"  -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppEigen/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
```

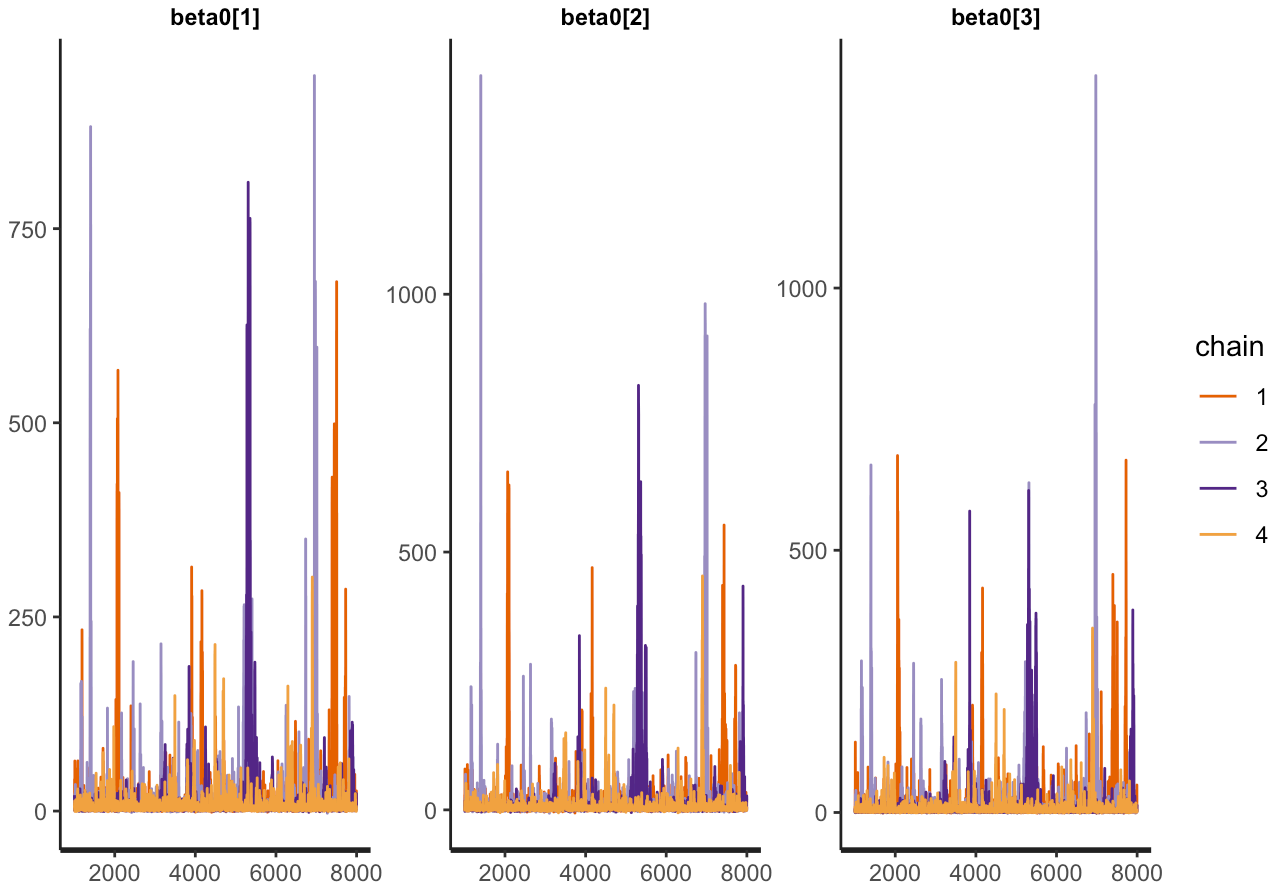
Partial Pooling #1 Version 2 (THIS WAS USED)

```
ppq_1_data_v2 <- list(N = length(final_curling$CompetitionName), # number of matches
                      k = length(unique(final_curling$CompetitionName)), # Number of competi
                      PowerPlayPresent = final_curling$PowerPlayPresent,
                      y = final_curling$USWin,
                      i = final_curling$CompetitionIndex, # for indexings sake
                      u0 = -0.30,
                      u1 = 0,
                      v0 = 1,
                      v1 = 1,
                      alpha0 = 1,
                      alpha1 = 1,
                      eta0 = 1,
                      eta1 = 1
)


set.seed(355)

fit_q1 <- stan(file = "partial_pooling_q1_v2.stan",
               data = ppq_1_data_v2,
               chains = 4,
               warmup = 1000,
               iter = 8000,
               control = list(adapt_delta = 0.999,
                              max_treedepth = 16))
```

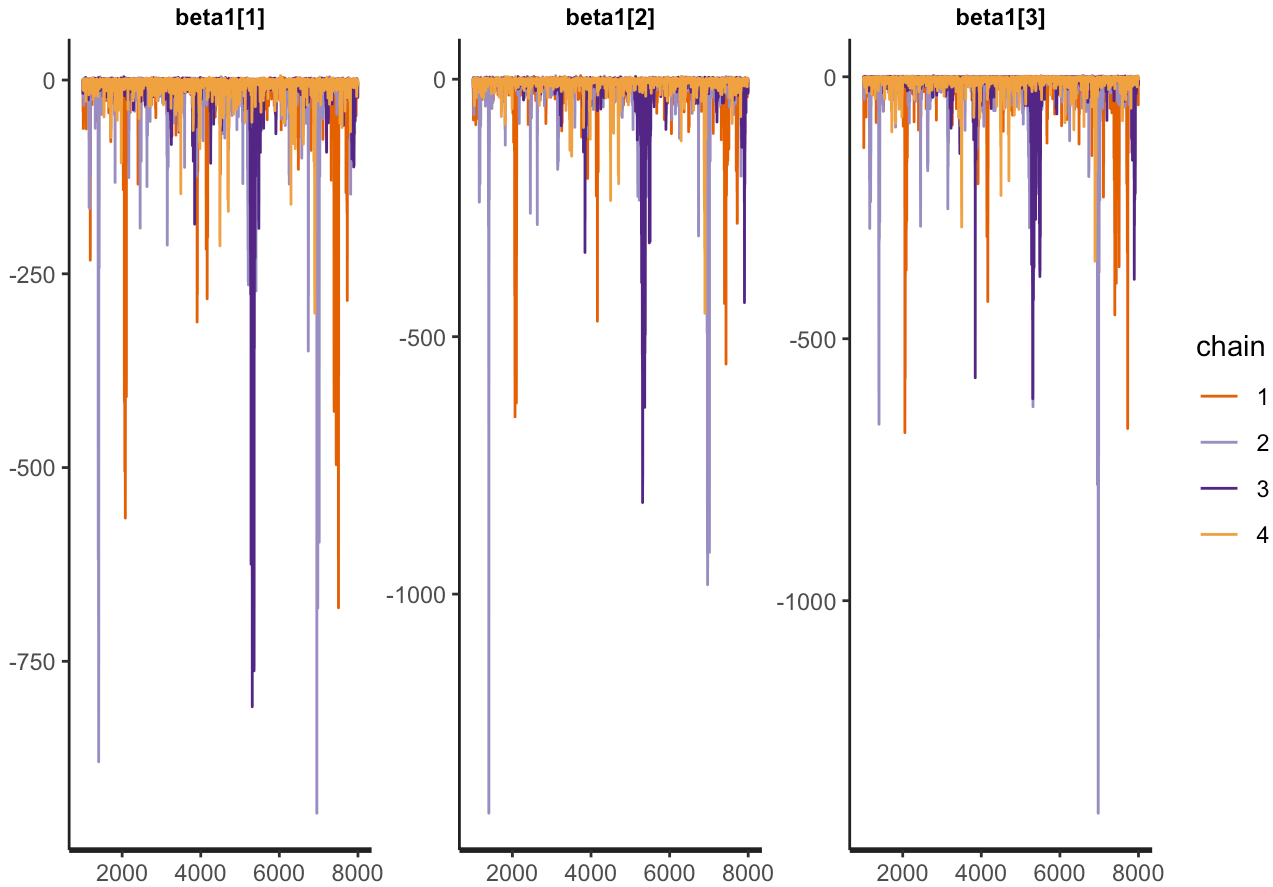
```
Trying to compile a simple C file

Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
using C compiler: ‘Apple clang version 17.0.0 (clang-1700.6.3.2)’
using SDK: ‘MacOSX26.2.sdk’
clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -
DNDEBUG   -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/Rcpp/include/"  -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppEigen/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppEigen/include/unsupported"  -
I"/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/BH/include" -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/StanHeaders/include/src/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/StanHeaders/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppParallel/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/rstan/include" -
DEIGEN_NO_DEBUG  -DBOOST_DISABLE_ASSERTS  -DBOOST_PENDING_INTEGER_LOG2_HPP  -
DSTAN_THREADS  -DUSE_STANC3 -DSTRICT_R_HEADERS  -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION  -
D_HAS_AUTO_PTR_ETC=0  -include '/Library/Frameworks/R.framework/Versions/4.5-
```

```
Chain 4: Iteration: 1001 / 8000 [ 12%]  (Sampling)
Chain 4: Iteration: 1800 / 8000 [ 22%]  (Sampling)
Chain 4: Iteration: 2600 / 8000 [ 32%]  (Sampling)
Chain 4: Iteration: 3400 / 8000 [ 42%]  (Sampling)
Chain 4: Iteration: 4200 / 8000 [ 52%]  (Sampling)
Chain 4: Iteration: 5000 / 8000 [ 62%]  (Sampling)
Chain 4: Iteration: 5800 / 8000 [ 72%]  (Sampling)
Chain 4: Iteration: 6600 / 8000 [ 82%]  (Sampling)
Chain 4: Iteration: 7400 / 8000 [ 92%]  (Sampling)
Chain 4: Iteration: 8000 / 8000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.469 seconds (Warm-up)
Chain 4:                8.081 seconds (Sampling)
Chain 4:                8.55 seconds (Total)
Chain 4:
```

```r
bulk_ess_beta0 <- vector(mode = "double", length = 3)
bulk_ess_beta1 <- vector(mode = "double", length = 3)

for (i in 1:3) {
  bulk_ess_beta0[i] <- ess_bulk(extract(fit_q1,
                                        pars = "beta0", permuted = FALSE)[,,i])
  bulk_ess_beta1[i] <- ess_bulk(extract(fit_q1,
                                        pars = "beta1", permuted = FALSE)[,,i])
}

rhat_beta0 <- vector(mode = "double", length = 3)
rhat_beta1 <- vector(mode = "double", length = 3)

for (i in 1:3) {
  rhat_beta0[i] <- Rhat(extract(fit_q1, pars = "beta0",
                                permuted = FALSE)[,,i])
  rhat_beta1[i] <- Rhat(extract(fit_q1, pars = "beta1",
                                permuted = FALSE)[,,i])
}



traceplot(fit_q1, pars = "beta0")
```




```
traceplot(fit_q1, pars = "beta1")
```




```r
print(fit_q1, pars = c("beta0", "beta1"))
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=8000; warmup=1000; thin=1;
post-warmup draws per chain=7000, total post-warmup draws=28000.

           mean se_mean    sd   2.5%   25%   50%   75% 97.5% n_eff Rhat
beta0[1]  12.83    1.76 39.51   0.64  2.54  4.35  9.04 77.59   507 1.01
beta0[2]  12.30    1.99 44.08  -1.26  1.19  3.07  7.73 90.92   491 1.01
beta0[3]  13.13    1.80 41.42   0.40  2.18  4.01  8.82 93.15   527 1.01
beta1[1] -11.48    1.76 39.53 -76.37 -7.71 -3.07 -1.19  1.05   507 1.01
beta1[2] -12.25    1.99 44.10 -90.73 -7.71 -3.06 -1.13  1.46   492 1.01
beta1[3] -13.36    1.81 41.46 -93.41 -9.13 -4.32 -2.34 -0.29   526 1.01

Samples were drawn using NUTS(diag_e) at Tue Apr  7 07:35:51 2026.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

```r
print(fit_q1, pars = c("exp_beta0", "exp_beta1"))
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=8000; warmup=1000; thin=1;
post-warmup draws per chain=7000, total post-warmup draws=28000.

             mean se_mean    sd 2.5%   25%   50%     75%        97.5% n_eff
exp_beta0[1]  Inf     NaN   NaN 1.89 12.62 77.14 8412.70 4.999849e+33   NaN
exp_beta0[2]  Inf     NaN   NaN 0.28  3.29 21.65 2265.80 3.069444e+39   NaN
exp_beta0[3]  Inf     NaN   NaN 1.49  8.86 55.34 6735.67 2.861822e+40   NaN
exp_beta1[1] 0.42    0.02  2.38 0.00  0.00  0.05    0.31 2.860000e+00 22517
exp_beta1[2] 0.79    0.06 10.24 0.00  0.00  0.05    0.32 4.290000e+00 27142
exp_beta1[3] 0.11    0.00  0.27 0.00  0.00  0.01    0.10 7.500000e-01 13177
             Rhat
exp_beta0[1]  NaN
exp_beta0[2]  NaN
exp_beta0[3]  NaN
exp_beta1[1]    1
exp_beta1[2]    1
exp_beta1[3]    1

Samples were drawn using NUTS(diag_e) at Tue Apr  7 07:35:51 2026.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

copy of stan file for printing purposes:

data { int<lower=1> N; // total number of observations int<lower=1> k; // number of matches int<lower=0, upper=1> PowerPlayPresent[N]; int<lower=0, upper=1> y[N]; int<lower=0> i[N]; // these are our indices for each match

// hyperparameters for regression coefficients real u0; real u1; real<lower=0> v0; real<lower=0> v1;

// hyperparameters for random effects real<lower=0> alpha0; real<lower=0> alpha1; real<lower=0> eta0; real<lower=0> eta1; }

parameters { // prior layer vector[k] beta0; vector[k] beta1;

// hyperprior layer real m0; real m1; real<lower=0> s0; real<lower=0> s1; }

transformed parameters { // mean model (log(odds)) vector[N] log_odds;

// probability vector[N] prob;

// fill in log_odds for each match for (l in 1:N) { log_odds[l] = beta0[i[l]] + beta1[i[l]] * PowerPlayPresent[l]; # betas depend on the group. i[l] tells you what group you're in }

// convert log odds to probability prob = inv_logit(log_odds);

// getting 1/s0^2 and 1/s1^2 for the Gamma hyperpriors real<lower=0> s02_inv; real<lower=0> s12_inv;

s02_inv = 1/(s0^2); s12_inv = 1/(s1^2); }

model { target += bernoulli_lpmf(y | prob); target += normal_lpdf(beta0 | m0, s0); target += normal_lpdf(beta1 | m1, s1); target += normal_lpdf(m0 | u0, v0); target += normal_lpdf(m1 | u1, v1); target += gamma_lpdf(s02_inv | alpha0, eta0); target += gamma_lpdf(s12_inv | alpha1, eta1); }

generated quantities { vector[k] exp_beta0; vector[k] exp_beta1; exp_beta0 = exp(beta0); exp_beta1 = exp(beta1); }

Partial Pooling for Q2.

Do these results (Odds of winning a curling match) vary by whether or not the play (power or otherwise) "failed in its intended result"? Beta hyperparameters need to be done properly $$ Y_{ij} | _{ij},_0, _1, *2 &Bernoulli(*{ij}) \ () &= _0 + _1 PowerPlay + *2 Success* | {ij} &= invlogit(_0 + *1 PowerPlay*{ij} + *2 Success*{ij}) \ B_0 &N(m_0, s_0^2) \ B_1 &N(m_1, s_1^2) \ B_2 &N(m_2, s_2^2) \

m_0 & (u_0, v_0^2) \ m_1 & (u_1, v_1^2) \ m_2 & (u_2, v_2^2) \

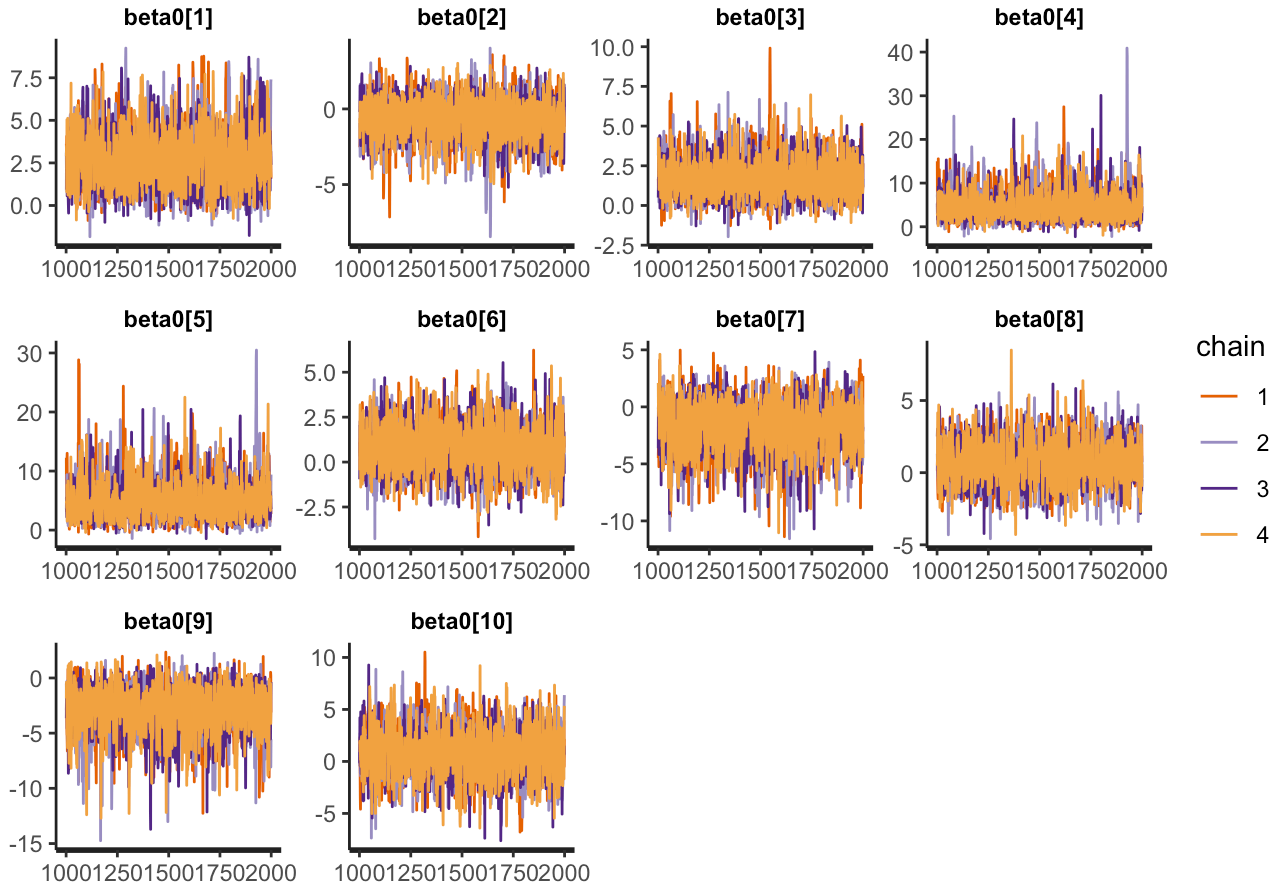
1 / s_0^2 & (_0, _0) \ 1 / s_1^2 & (_1, _1) \ 1 / s_2^2 & (_2, _2) \

$$

```
#
#   int<lower=1> N; // total number of observations - sets
#   int<lower=1> k; // number of matches
#   vector[N] x; // this is powerplay
#   vector[N] x2; // this is success
#   int<lower=0, upper=1> y[N];
#   int<lower=0> i[N]; // these are our indices for each Match
#                      // We need right here our two indicator variables
#
#   // hyperparameters for regression coefficients (Betas) and likelihood variance
#   real u0;
#   real u1;
#   real u2;
#   real<lower=0> v0;
#   real<lower=0> v1;
#   real<lower=0> v2;
#
#
#   // hyperparameters for random effects (Gamma funcs)
#   real<lower=0> alpha0;
#   real<lower=0> alpha1;
#   real<lower=0> alpha2;
#
#   real<lower=0> eta0;
#   real<lower=0> eta1;
#   real<lower=0> eta2;

# curlin <- curlin %>% mutate(Points = ifelse((Points > 2), 1, 0))
```

```r
PPQ2Data <- list(x = curlin$PowerPlay,
                 x2 = curlin$Points,
                 y = curlin$USWin,
                 N = length(curlin$CompetitionName), #sets
                 k = length(unique(curlin$MatchID)),# Number of matches
                 i = curlin$MatchID, # for indexings sake
                 u0 = 1,
                 u1 = 1,
                 u2 = 1,

                 v0 = 1,
                 v1 = 1,
                 v2 = 1,

                 alpha0 = 1,
                 alpha1 = 1,
                 alpha2 = 1,

                 eta0 = 1,
                 eta1 = 1,
                 eta2 = 1

)

fit <- stan(file = "PPQ2.stan",
            data = PPQ2Data,
            chains = 4,
            warmup = 1000,
            iter = 2000)
```

```
Trying to compile a simple C file

Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
using C compiler: ‘Apple clang version 17.0.0 (clang-1700.6.3.2)’
using SDK: ‘MacOSX26.2.sdk’
clang -arch arm64 -std=gnu2x -I"/Library/Frameworks/R.framework/Resources/include" -
DNDEBUG   -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/Rcpp/include/"  -I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppEigen/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppEigen/include/unsupported"  -
I"/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/BH/include" -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/StanHeaders/include/src/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/StanHeaders/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-
arm64/Resources/library/RcppParallel/include/"  -
I"/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library/rstan/include" -
DEIGEN_NO_DEBUG  -DBOOST_DISABLE_ASSERTS  -DBOOST_PENDING_INTEGER_LOG2_HPP  -
```




```
intercepts <- summary(fit, pars = c('beta0'))$summary[,1]
slopes <- summary(fit, pars = c('beta1'))$summary[,1]
slopes2 <- summary(fit, pars = c('beta2'))$summary[,1]

slopes[1] + intercepts[1] + slopes2[1]# why si this bigger than 1
```

```
beta1[1]
2.201019
```

```
mean(intercepts)
```

```
[1] 1.675605
```

```
mean(slopes)
```

```
[1] -0.07722404
```

```
  curlin %>%   ggplot(aes(PowerPlay, USWin)) +
  geom_point(col = "white") +
  theme_classic() +
  geom_abline(slope = slopes, intercept = intercepts, col = "grey")
```